JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Base64.Decoder

java.lang.Object
    java.util.Base64.Decoder

Enclosing class:

Base64

```
public static class Base64.Decoder
extends Object
```

This class implements a decoder for decoding byte data using the Base64 encoding scheme as specified in RFC 4648 and RFC 2045.

The Base64 padding character `'='` is accepted and interpreted as the end of the encoded byte data, but is not required. So if the final unit of the encoded byte data only has two or three Base64 characters (without the corresponding padding character(s) padded), they are decoded as if followed by padding character(s). If there is a padding character present in the final unit, the correct number of padding character(s) must be present, otherwise `IllegalArgumentException` (`IOException` when reading from a Base64 stream) is thrown during decoding.

Instances of `Base64.Decoder` class are safe for use by multiple concurrent threads.

Unless otherwise noted, passing a `null` argument to a method of this class will cause a `NullPointerException` to be thrown.

If the decoded byte output of the needed size can not be allocated, the decode methods of this class will cause an `OutOfMemoryError` to be thrown.

Since:

1.8

See Also:

`Base64.Encoder`

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
`byte[]`	**`decode`**`(byte[] src)`	Decodes all bytes from the input byte array using the `Base64` encoding scheme, writing the results into a

		newly-allocated output byte array.
int	**decode**(byte[] src, byte[] dst)	Decodes all bytes from the input byte array using the Base64 encoding scheme, writing the results into the given output byte array, starting at offset 0.
byte[]	**decode**(String src)	Decodes a Base64 encoded String into a newly-allocated byte array using the Base64 encoding scheme.
ByteBuffer	**decode**(ByteBuffer buffer)	Decodes all bytes from the input byte buffer using the Base64 encoding scheme, writing the results into a newly-allocated ByteBuffer.
InputStream	**wrap**(InputStream is)	Returns an input stream for decoding Base64 encoded byte stream.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

decode

```
public byte[] decode(byte[] src)
```

Decodes all bytes from the input byte array using the Base64 encoding scheme, writing the results into a newly-allocated output byte array. The returned byte array is of the length of the resulting bytes.

Parameters:

`src` - the byte array to decode

Returns:

A newly-allocated byte array containing the decoded bytes.

Throws:

`IllegalArgumentException` - if `src` is not in valid Base64 scheme

decode

```
public byte[] decode(String src)
```

Decodes a Base64 encoded String into a newly-allocated byte array using the Base64 encoding scheme.

An invocation of this method has exactly the same effect as invoking `decode(src.getBytes(StandardCharsets.ISO_8859_1))`

Parameters:

`src` - the string to decode

Returns:

A newly-allocated byte array containing the decoded bytes.

Throws:

`IllegalArgumentException` - if `src` is not in valid Base64 scheme

decode

```
public int decode(byte[] src,
                  byte[] dst)
```

Decodes all bytes from the input byte array using the Base64 encoding scheme, writing the results into the given output byte array, starting at offset 0.

It is the responsibility of the invoker of this method to make sure the output byte array `dst` has enough space for decoding all bytes from the input byte array. No bytes will be written to the output byte array if the output byte array is not big enough.

If the input byte array is not in valid Base64 encoding scheme then some bytes may have been written to the output byte array before IllegalargumentException is thrown.

Parameters:

`src` - the byte array to decode

`dst` - the output byte array

Returns:

The number of bytes written to the output byte array

Throws:

`IllegalArgumentException` - if `src` is not in valid Base64 scheme, or `dst` does not have enough space for decoding all input bytes.

decode

`public ByteBuffer decode(ByteBuffer buffer)`

Decodes all bytes from the input byte buffer using the `Base64` encoding scheme, writing the results into a newly-allocated ByteBuffer.

Upon return, the source buffer's position will be updated to its limit; its limit will not have been changed. The returned output buffer's position will be zero and its limit will be the number of resulting decoded bytes

`IllegalArgumentException` is thrown if the input buffer is not in valid Base64 encoding scheme. The position of the input buffer will not be advanced in this case.

Parameters:

`buffer` - the ByteBuffer to decode

Returns:

A newly-allocated byte buffer containing the decoded bytes

Throws:

`IllegalArgumentException` - if `buffer` is not in valid Base64 scheme

wrap

`public InputStream wrap(InputStream is)`

Returns an input stream for decoding `Base64` encoded byte stream.

The `read` methods of the returned `InputStream` will throw `IOException` when reading bytes that cannot be decoded.

Closing the returned input stream will close the underlying input stream.

Parameters:

`is` - the input stream

Returns:

the input stream for decoding the specified Base64 encoded byte stream

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.